



SAPIENZA
UNIVERSITÀ DI ROMA

MASTER'S DEGREE IN ARTIFICIAL INTELLIGENCE AND ROBOTICS

**Beyond geometry: semantic hazard
detection via 3D scene graphs for safe
Human-Robot Interaction**

HUMAN-ROBOT-AI INTERACTION

Professors:

Iocchi Luca
Suriani Vincenzo

Students:

Musumeci Federica*
Ricci Andrea*
Ficarola Filippo*

Academic Year 2025/2026

* All the students contributed equally to the project.

Abstract

As autonomous robots become increasingly integrated into unstructured domestic environments, ensuring Safe Human-Robot Interaction (HRI) requires moving beyond traditional geometric collision avoidance toward a deeper semantic understanding of physical hazards. This project presents a novel, Scene Graph-based anomaly detection framework designed to proactively filter unsafe user commands. Leveraging the Stanford 3D Scene Graph dataset, which represents households as relational structures, the system can assess whether a requested action is contextually hazardous based on the spatial and physical relationships between objects.

The proposed framework utilizes a Relational Graph Autoencoder (RGAE) based on the GraphSAGE architecture. To enable the model to reason about physical compatibility, object nodes are represented through a combination of semantic embeddings and augmented physical attributes (such as flammability and electrical sensitivity). To address the lack of labeled safety data in robotics, a Large Language Model (LLM) was employed as a "knowledge oracle" to automatically label thousands of potential interactions, establishing a baseline of domestic normality. The RGAE was trained exclusively on safe environment states, learning to detect hazardous anomalies (e.g., placing electronics near water or heat sources) by analyzing the topological reconstruction error of the requested interaction. Finally, the proposed architecture was validated on a real TIAGo robot in a laboratory environment: before executing a command, the robot virtually inserts the requested interaction into the room's scene graph, queries the detector, and only then selects among three behaviors: autonomous approval, human-in-the-loop validation, or automated action blocking. This approach delivers high predictive accuracy and essential explainability, fostering user trust in home-care robotics.

Contents

1	Introduction	3
1.1	Context and motivation	3
1.2	Problem statement	4
1.3	Project objectives	5
1.4	Document outline	6
2	Related works	7
2.1	3D scene graphs as an environment representation	7
2.2	Anomaly detection for robotic safety	8
2.3	Large Language Models as sources of physical and commonsense knowledge in embodied AI	8
2.4	Summary and research gap	9
3	Methodology and system architecture	10
3.1	Pipeline overview	10
3.2	Data extraction from the Stanford 3D Scene Graph	10
3.3	Candidate interaction generation	12
3.4	Hybrid node representation	14
3.5	LLM-based safety labeling	16
3.6	Relational Graph Autoencoder	18
3.7	Anomaly scoring and decision logic	20
4	Experimental analysis and offline results	23
4.1	Training setup	23
4.2	Anomaly score evaluation	24
4.3	Explainability and spatial visualization	25
4.4	Limitations of the offline model	27
5	Integration into a Human-Robot Interaction loop	29
5.1	From an offline detector to an embodied system	29
5.2	System architecture and information flow	29
5.3	From speech to a structured query	30

5.4	Context-aware access control	31
5.5	Verbal hazard explanation	32
5.6	Example interactions	32
6	Conclusions and future work	36
	Bibliography	38

Chapter 1

Introduction

1.1 Context and motivation

The landscape of robotics has undergone a fundamental paradigm shift over the last decade. Robotic agents are no longer confined to the highly structured, deterministic environments of industrial manufacturing floors. Instead, they are increasingly deployed in dynamic, unstructured spaces such as hospitals, public facilities, and private homes. In these scenarios, the robot is no longer an isolated machine operating behind a safety fence; it is a collaborator embedded in a shared living space, acting on instructions issued in natural language by non-expert users.

This transition brings the field of Human-Robot Interaction (HRI) to the forefront, with safety emerging as the most critical requirement. Classical robot safety has been dominated by geometric and kinematic concerns: avoiding collisions, limiting joint torques, respecting protective separation distances, and stopping when a human enters the workspace. These mechanisms are necessary, but they address only *how* the robot moves - not *what* it is being asked to do. As robots evolve from navigational agents into active manipulators of the household, a complementary form of safety becomes essential: the ability to anticipate the consequences of an action on the state of the world before executing it.

The need for such anticipation is amplified by the human side of the interaction. Domestic users cannot be assumed to issue only sensible commands. Instructions may be careless, ambiguous, mistaken, or, in the case of children, issued without any understanding of their consequences. A robot that blindly executes every syntactically valid request effectively transfers the full burden of hazard assessment onto the user, which is precisely the opposite of what an assistive system should do. For home-care robotics to be trusted and adopted, the robot itself must possess a degree of semantic safety awareness: the ability to recognize when the state of the world it is about to create is a dangerous one, and to refuse or question commands accordingly.

1.2 Problem statement

Despite significant advancements in Simultaneous Localization and Mapping (SLAM) and 3D vision, traditional robotic frameworks still perceive the world largely through a geometric lens. A robot may possess a high-fidelity point cloud or a volumetric map of a room, allowing it to know precisely where surfaces and objects are located. However, it often lacks the commonsense reasoning to understand what those objects represent in relation to one another.

This creates a dangerous gap between geometric feasibility and semantic hazard awareness. For example, a motion planner might compute a perfectly valid, collision-free trajectory to place a plugged-in toaster on the edge of a wet sink. Geometrically, the action is flawless (the spatial constraints are met); physically, it is a catastrophic hazard. The danger lies not in the motion, but in the resulting configuration of the world.

Closing this gap is not a matter of simply detecting statistical outliers. Standard anomaly detection models often confuse “unusual” with “unsafe”. A pillow placed inside a refrigerator is a rare, anomalous event, but it poses no physical danger. Conversely, placing a highly flammable item near an active stove might be a statistically common configuration in a messy kitchen, yet it represents a severe risk. Any practical detector must therefore target genuine physical hazard rather than mere statistical rarity, so that it does not degenerate into an over-restrictive assistant that refuses legitimate requests.

The core problem addressed in this report can therefore be stated as follows: given a structured representation of an indoor scene and a user command that would insert a new object into that scene through a specific spatial relation, decide automatically whether the resulting scene state constitutes a physical hazard. The spatial relations considered are three primitive placement interactions (*placed_near*, *placed_inside*, and *placed_on_top*) which cover a wide range of everyday manipulation requests.

Two additional constraints shape the problem. First, labeled safety data for domestic robotics essentially does not exist: there is no large-scale dataset annotating which object-object-context combinations are dangerous, so the supervision signal itself must be constructed. Second, safety assessment cannot remain a purely offline classification exercise: to be useful in HRI, the detector must be embedded in the robot’s decision loop, so that the evaluation happens between the reception of a command and its execution. Moreover, the appropriate response is not a property of the action alone but of the action-user pair: a borderline request may deserve a simple warning when issued by an adult, but a firm refusal when issued by a child. The robot’s decision must therefore be both user-aware and communicated in an understandable, trust-preserving way.

1.3 Project objectives

The main objective of this project is to design, implement, and validate an innovative, scene graph-based anomaly detection framework that empowers service robots to comprehend, reason about, and prevent hazardous physical interactions within domestic environments. To achieve this goal, the project targets the following specific objectives:

1. **Construct a relational dataset of object interactions from real indoor scenes.** Starting from three environments of the Stanford 3D Scene Graph dataset, extract objects with their semantic categories, materials, volumes, and affordances; group them by room and systematically generate candidate interactions using geometric checks (relative dimensions and volumes) to discard physically impossible placements. Both intra-room configurations and cross-room object transfers are considered, so that the dataset covers the kinds of relocations a user may realistically request.
2. **Design a hybrid node representation combining semantics and physics.** Represent each object through the concatenation of a sentence-level semantic embedding and a compact danger profile encoding physically relevant attributes (such as flammability, electrical sensitivity, and toxicity), so that the learning model can reason about physical compatibility rather than lexical similarity alone.
3. **Generate ground-truth safety labels through an LLM used as a knowledge oracle.** In the absence of annotated safety data, employ a Large Language Model to label thousands of candidate interactions as safe or hazardous under an explicit rubric that isolates physical danger from mere contextual strangeness, effectively distilling human common-sense safety knowledge into a training signal.
4. **Develop a Relational Graph Autoencoder (RGAE) for anomaly detection.** Train an inductive graph neural network, built on GraphSAGE convolutions with a relation-predicting decoder, exclusively on safe scene states, so that it internalizes a model of “domestic normality.” At inference time, a requested interaction is virtually inserted into the room graph, and its anomaly score is computed from the reconstruction error of the predicted spatial relation.
5. **Validate the detector offline and integrate it into a simulated HRI loop.** Quantitatively evaluate the model on held-out safe and dangerous interactions, and then transfer the pipeline to a real TIAGo robot operating in a laboratory environment. There, a virtual-insertion paradigm is adopted: the robot updates the scene graph hypothetically, queries the detector, and only then selects among three behaviors (autonomous approval, human-in-the-loop confirmation, or automatic refusal) with the additional capability of adapting its permissiveness to the user profile (e.g., responding more restrictively to a child than to an adult).

1.4 Document outline

The remainder of this report is organized as follows.

Chapter 2 reviews the relevant literature, covering 3D scene graphs as an environment representation (with particular attention to the Stanford 3D Scene Graph dataset), anomaly detection approaches for robotic safety, and the emerging use of Large Language Models as sources of physical and common-sense knowledge in embodied AI.

Chapter 3 presents the methodology and system architecture in detail: the data extraction and geometric filtering pipeline, the construction of the hybrid semantic-physical node features and danger profiles, the generation of intra-room and cross-room relations, the LLM-based labeling procedure, the architecture and training of the Relational Graph Autoencoder, and the anomaly-scoring and thresholding logic.

Chapter 4 reports the offline experimental evaluation, including the training setup, the behavior of the anomaly score under safe and hazardous insertions, an explainability analysis relating the anomaly score to the physical attributes of the objects involved, and a critical discussion of the limitations of the offline model.

Chapter 5 describes the integration of the detector on a real TIAGo robot, the virtual-insertion decision paradigm, and the context-aware access control policy that differentiates between adult and child users through the three-tier response scheme.

Chapter 6 concludes the report by summarizing the results achieved and outlining future developments, including the extension of the framework to a richer set of spatial predicates.

Chapter 2

Related works

This chapter surveys the three areas of research on which the proposed system rests. The first is the representation of indoor environments as 3D scene graphs, with particular attention to the Stanford 3D Scene Graph dataset, which supplies the relational description of the home that the system operates on. The second is anomaly detection for robotic safety, which is the task the system performs. The third is the use of Large Language Models (LLMs) as a source of physical and commonsense knowledge for embodied agents. The aim of this review is to explain what each area offers, to show where the state of the art still falls short, and to make clear the gap that the rest of the report sets out to close.

2.1 3D scene graphs as an environment representation

A scene graph describes a physical space as a set of nodes that stand for entities in the environment and a set of edges that capture the spatial and hierarchical relationships between them. Armeni and colleagues introduced this idea for buildings, organising each environment into layered nodes for the building, its rooms, and the objects those rooms contain, with edges that connect the layers and record how the entities relate [2]. The attraction of the representation is that it separates the geometry of a place from its meaning. Two rooms with very different shapes can share the same semantic structure, and a robot that reasons on the graph can ask questions about what objects are and how they are arranged rather than only about the geometric layout of the environment.

The Stanford 3D Scene Graph dataset accompanies this formulation and provides semantic annotations for a collection of real indoor environments, organised as residential units in which each node carries attributes such as its semantic class. A widely used preprocessed version of the dataset adds edges that connect nearby objects within the same room, which turns each room into a small relational graph of the objects it holds. This object level connectivity is what makes the dataset suitable for reasoning about how objects sit together, and it is the level at which the present work operates.

Since its introduction, the representation has been adopted broadly across robotics, where it has been extended to dynamic scenes with moving agents and to systems that build such graphs online from sensor data. In practice, scene graphs are primarily used to support spatial perception, semantic mapping, and long-horizon task planning, enabling robots to represent their surroundings in a structured and semantically meaningful way. Most existing approaches exploit this representation to reason about the environment itself - for example, to localize objects, understand room layouts, or plan navigation and manipulation tasks. Much less attention has been devoted to using scene graphs as a representation for safety reasoning, where the goal is not to determine where an object is, but whether a particular relationship between two objects results in a safe or hazardous state.

2.2 Anomaly detection for robotic safety

Anomaly detection identifies states that deviate from a model of what is normal, and a common and effective way to build such a model is to train a reconstruction based method, such as a graph autoencoder, on normal data alone and then to flag inputs that the model reconstructs poorly. Such graph based detectors are mature and have been widely investigated [3], but they have been applied mostly to domains such as fraud, network intrusion, and surveillance, where the graphs describe transactions or traffic rather than the physical layout of a home.

Within robotics, interest has grown in agents that notice danger rather than only carrying out chores. Among the most relevant works is the framework proposed by Song et al. [5], which employs a Large Language Model to inspect household scenes and infer potential hazards arising from object arrangements.

Two difficulties recur across this literature. The first is that a detector tuned to flag whatever is statistically rare does not, by itself, track danger, since some rare arrangements are perfectly safe while some hazardous ones are entirely common. A detector meant for safety therefore has to be steered toward genuine physical risk rather than mere novelty. The second is that most systems reason about objects in isolation, typically starting from images or textual scene descriptions. As a result, the interaction between objects, which is often where domestic hazards arise, is not explicitly modeled.

2.3 Large Language Models as sources of physical and commonsense knowledge in embodied AI

Geometric planners can compute where a robot may move but carry no sense of what the objects around it mean or how they behave, and this missing world knowledge is precisely what large language models supply. Because they are trained on vast amounts

of human text, these models hold a broad understanding of everyday objects, and the field has increasingly used them to inject that understanding into embodied systems. A key insight from this line of research is that language models must be grounded in the robot’s current environment, as reasoning based solely on linguistic knowledge may lead to actions that appear plausible but are physically infeasible or unsafe [1].

ConceptBot, proposed by Leanza and colleagues, is a representative and closely related example [4]. It combines a Large Language Model with a knowledge graph enriched using information from ConceptNet, enabling it to reason not only about the hazards associated with individual objects but also about the risks arising from their interactions. In their evaluation this capacity for risk aware reasoning was the sharpest advantage over an earlier grounding baseline that lacked it, which confirms that the safety of object interactions is a live and competitive research problem. The system of Song [5] discussed above shares the same underlying choice, since it too consults a language model to reason about hazards.

What these approaches have in common is that the language model sits inside the decision loop and is queried at run time, once for every request. This is powerful, because the model generalises to situations never seen before, but it also makes each decision depend on a heavy and often remote model, with the cost, latency, and nondeterminism that this implies. A less explored alternative is to consult the language model offline, so that its commonsense is captured once and then reused. This shift, from consuming commonsense at run time to distilling it in advance, is central to the direction taken in this report.

2.4 Summary and research gap

The three areas reviewed here are individually mature, yet they leave a specific gap at their intersection. Scene graphs give a clean relational description of a home, but they have served mainly for perception and planning rather than for judging the safety of the arrangements they encode. Anomaly detection offers effective tools for spotting deviations from normality, but in the home setting it has rarely been made relational and rarely been steered toward physical danger rather than statistical rarity. Large language models bring the commonsense that geometric systems lack, but they are typically kept in the run time loop, which makes them costly to rely on aboard a real robot. The proposed framework is positioned exactly at this intersection, and the chapters that follow describe how it turns the relational structure of a scene graph, a model of normality learned from safe states, and commonsense distilled offline from a language model into a single detector that decides, before an action is taken, whether a requested placement would make a scene unsafe.

Chapter 3

Methodology and system architecture

3.1 Pipeline overview

This chapter describes the design and implementation of the proposed anomaly detection framework. The overall pipeline, summarized in Figure 3.1, consists of five stages. First, object-level information is extracted from three environments of the Stanford 3D Scene Graph dataset and organized by room (Section 3.2). Second, candidate object-object interactions are systematically generated and filtered through geometric feasibility checks (Section 3.3). Third, each object is encoded through a hybrid representation that combines a semantic sentence embedding with a compact physical danger profile (Section 3.4). Fourth, a Large Language Model is employed as a knowledge oracle to label every candidate interaction as safe or hazardous (Section 3.5). Finally, a Relational Graph Autoencoder is trained exclusively on safe interactions (Section 3.6) and used at inference time to score requested actions and map them to a three-tier decision policy (Section 3.7).

3.2 Data extraction from the Stanford 3D Scene Graph

The starting point of the pipeline is the Stanford 3D Scene Graph dataset, which provides a hierarchical, semantically annotated representation of real indoor environments reconstructed from 3D scans. Each environment is organized as a four-layer hierarchy (building, rooms, objects, cameras); for the purposes of this project, only the room and object layers are relevant. In particular, three residential environments were selected, providing a heterogeneous mix of room types (kitchens, bathrooms, bedrooms, living rooms, and home offices) and of the everyday objects they contain.

Each scene is stored in a file containing two dictionaries of interest: an *object* dictionary, mapping each object instance to its annotated properties, and a *room* dictionary,

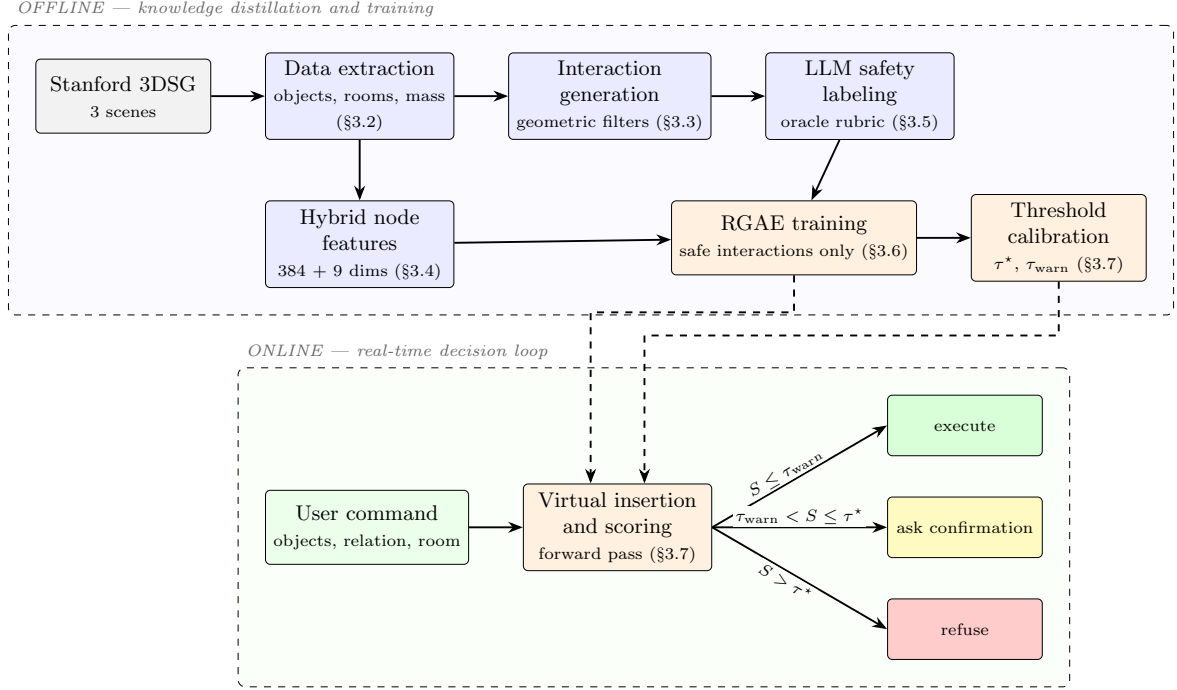


Figure 3.1: Overview of the proposed pipeline. In the offline phase, interactions extracted from the Stanford 3D Scene Graph are labeled by an LLM oracle and used to train the RGAE on safe configurations only; in the online phase, a requested action is scored with a single forward pass and mapped to one of three responses. The LLM is consulted only offline: run-time decisions do not depend on it.

describing the rooms of the building. For every object, the following fields are extracted:

- **Semantic class** (e.g., *toaster*, *sink*, *book*), which identifies the object category;
- **Material** (e.g., *metal*, *ceramic*, *paper*), which is essential for reasoning about physical hazards such as flammability or fragility;
- **Geometric properties**, namely the volume and the dimensions of the object’s bounding box, later used to validate the physical feasibility of placement interactions;
- **Action affordances** (e.g., *turn on*, *cut*, *eat*), which describe the typical ways a human can interact with the object and carry implicit information about its function;
- **Parent room**, which links the object to the room containing it.

Each object stores only the numeric identifier of the room that contains it; the corresponding room category (e.g., *kitchen*, *bathroom*) is retrieved from the room dictionary, and objects are then grouped by category. This room-level grouping is the structural

backbone of the entire framework: each room becomes an independent graph, and every candidate interaction is always evaluated within the context of a specific room category.

Since the annotations originate from real-world scans, the raw data requires a normalization step. Material annotations may be missing or provided as lists of multiple values, in which case they are concatenated into a single descriptor and missing entries are mapped to a conventional *unknown* value; missing volumes are treated as zeros, and all textual fields are lowercased to ensure consistent matching across scenes.

Two further processing steps complete the extraction stage. First, since the same room may contain several instances of essentially identical objects (e.g., multiple chairs of the same kind), a deduplication step removes redundant instances: two objects are considered duplicates when they share the same semantic class, material, volume, and affordance set. This prevents the interaction generation stage from producing large numbers of near-identical samples that would bias the dataset without adding information.

Second, the dataset does not annotate object masses, which are however relevant to safety reasoning (e.g., a heavy object crushing a fragile one). Mass is therefore estimated from the available annotations: each material is associated with a representative density (e.g., approximately 5000 kg/m^3 for metallic objects, 600 kg/m^3 for wood, 200 kg/m^3 for fabric), and the mass of an object is computed as the product of its density and its annotated volume. While such estimates are necessarily approximate (the density of a composite object is approximated by that of its primary material) they are sufficient to support the binary “heavy object” attribute used in the physical representation of Section 3.4.

The output of this stage is, for each of the three scenes, a set of rooms, each containing a list of unique objects described by their semantic class, material, volume, bounding-box dimensions, estimated mass, and affordances. These per-room object lists are the input to the interaction generation process described in the next section.

3.3 Candidate interaction generation

The environments extracted so far describe rooms in their current state. However, the goal of the framework is to evaluate *requested* actions, i.e., placements that do not exist yet. The second stage of the pipeline therefore generates a large set of candidate interactions between object pairs, which will later be labeled (Section 3.5) and used to train and evaluate the model.

Interaction primitives

Three placement primitives are considered, chosen to represent the most common manipulation requests a user can issue to a domestic robot:

- *placed_near*: object A is positioned in the immediate vicinity of object B (e.g., a toaster placed near a sink);
- *placed_inside*: object A is inserted into object B (e.g., clothes placed inside a washing machine);
- *placed_on_top*: object A is placed on the upper surface of object B (e.g., a book placed on a desk).

Each interaction is directional: the first object is the one being manipulated, while the second acts as the reference or destination.

Geometric feasibility filters

Not every primitive is physically possible for every pair of objects: a sofa cannot be placed inside a microwave. To avoid populating the dataset with physically impossible configurations, each candidate pair is passed through simple geometric checks based on the bounding-box dimensions and volumes extracted in Section 3.2:

- *placed_near* is always considered feasible, since proximity imposes no size constraint;
- *placed_inside* is generated only if the largest bounding-box dimension of A is smaller than the largest dimension of B. This is a deliberately simple test: it is a necessary but not sufficient condition for containment (it does not compare the remaining dimensions, nor does it account for the internal capacity of B, which is not annotated in the dataset). Its role is only to remove evidently impossible insertions, such as a sofa inside a microwave;
- *placed_on_top* is generated only if the volume of A does not exceed 2.5 times the volume of B. Unlike containment, support is not strictly bounded by size - a large, light object can legitimately rest on a smaller one (a folded blanket on a stool) - so a hard geometric limit would be wrong here. The threshold instead cuts off only extreme disproportions (a wardrobe on a cup), where the configuration would be physically unstable regardless of the objects involved.

These filters are intentionally permissive, and the asymmetry between them is deliberate. Their purpose is not to judge whether a placement is safe, but only to discard configurations that could not physically occur and that a robot would reject at the motion-planning level anyway. Physically possible but potentially harmful placements, such as a heavy object resting on a fragile one, deliberately pass the filters and reach the labeling stage, so that the distinction between safe and hazardous states is learned by the model rather than hard-coded in geometric rules.

Intra-room and cross-room scenarios

Candidate interactions are generated in two complementary settings.

In the **intra-room** setting, all pairs of objects belonging to the same room are combined, and every primitive that passes the geometric filters produces a candidate interaction. These samples represent rearrangements within a room, and the room category (e.g., *kitchen*) is attached to each sample as its context.

In the **cross-room** setting, objects from one room are paired with objects from a different room, simulating requests in which the user asks the robot to move an object from its original location to another room (e.g., “take the hair dryer to the bathroom”). In this case, the context associated with the sample is the *destination* room, since that is where the resulting configuration would exist. Cross-room pairings grow combinatorially with the number of objects; to keep the dataset balanced and the labeling cost manageable, a random subset of at most 1500 cross-room interactions is retained per scene.

Resulting dataset

The interactions generated from the three scenes are merged into a single dataset, where each entry records the two objects with their physical properties (class, material, volume, affordances), the interaction primitive, and the room context. Duplicate entries are removed and the dataset is shuffled to avoid ordering effects. The final dataset contains 8258 candidate interactions, of which 3758 intra-room and 4500 cross-room.

3.4 Hybrid node representation

To be processed by a neural network, each object must be converted into a numerical feature vector. The choice of this representation is critical: to recognize that a hair dryer and a bathtub form a dangerous pair, the model needs to know not only what the objects are, but also which physical properties they have. The proposed representation therefore combines two complementary components: a semantic embedding, which captures what the object is, and a physical danger profile, which captures how the object can cause or suffer harm.

Semantic embedding

Each object is first described by a short natural-language string combining its class and material (e.g., “toaster made of metal”, “book made of paper”). This string is encoded with a pre-trained sentence embedding model (*all-MiniLM-L6-v2*, from the Sentence-Transformers family), which maps it to a 384-dimensional vector. Sentence embeddings place semantically related concepts close to each other in the vector space: a toaster and a microwave receive similar representations, as do a towel and a blanket. This provides

the model with general knowledge about object categories and brings a practical benefit: since the embedding model is pre-trained on natural language, it is not restricted to the objects present in the three scenes. Any new object requested by a user can be encoded in the same way, simply by describing its name and material, and it will be placed near similar known objects in the embedding space.

Danger profile

Semantic similarity alone, however, is not sufficient for safety reasoning, and can even be misleading. Two objects may be lexically close but physically very different with respect to hazards (a plastic toy knife and a real one), while objects that are lexically distant may share the same critical property (a newspaper and a silk curtain are both flammable). Safety depends on a small set of physical attributes that the embedding does not represent explicitly.

For this reason, each object is additionally described by a *danger profile*: a nine-dimensional binary vector in which each component flags a physically relevant property. The nine attributes are: *electrical*, *flammable*, *sharp*, *toxic*, *water-sensitive*, *heavy*, *edible*, *heat source*, and *fragile*. Together, they cover the main mechanisms by which domestic accidents occur: electric shock, fire, cuts, poisoning and contamination, water damage, crushing, and breakage.

The attributes are assigned through simple rule-based heuristics that inspect the object class, its material, and its affordances. For example, an object is marked as *electrical* if its class or affordances match a list of appliance-related keywords (e.g., *lamp*, *microwave*, *turn on*); as *flammable* if its material belongs to combustible categories such as paper, fabric, or wood; as a *heat source* if it is a cooking or heating appliance. Seven of the nine attributes are assigned by keyword matching. The remaining two are computed from other information: *water-sensitive* is activated whenever the object is electrical or made of paper, and *heavy* is activated when the mass estimated in Section 3.2 exceeds a fixed threshold of 2 kg.

This rule-based approach is intentionally simple and, as such, has two limitations. First, keyword matching may occasionally mislabel unusual objects whose name or material does not follow common patterns. Second, each attribute is a simple binary flag: it records *whether* an object has a hazardous property, but not *how severe* that property is. A match and a canister of gasoline, for instance, are both marked as flammable, even though the risks they pose are very different. A continuous encoding of hazard intensity would capture such differences, and represents a natural refinement of the profile. Nevertheless, the binary formulation has clear advantages: it is transparent, requires no training data, can be extended by simply adding keywords or attributes, and injects exactly the kind of information the semantic embedding lacks, in a form that is directly interpretable - a property that will be exploited in the explainability analysis of Chapter

Final node features

The final representation of an object is the concatenation of its 384-dimensional semantic embedding and its 9-dimensional danger profile, yielding a 393-dimensional feature vector. In this hybrid space, two objects are close when they are both semantically similar *and* physically compatible, which is precisely the notion of similarity that safety reasoning requires. These vectors constitute the node features of the room graphs on which the model of Section 3.6 operates.

3.5 LLM-based safety labeling

The interaction generation stage produces thousands of candidate placements, but no information about which of them are dangerous. As discussed in Section 1.2, no large-scale dataset of this kind exists, and manually annotating every interaction would be prohibitively time-consuming and inevitably inconsistent across annotators. At the same time, judging whether “a newspaper placed on top of a stove in a kitchen” is dangerous does not require specialized expertise: it requires common sense. This observation motivates the use of a Large Language Model as a *knowledge oracle*: LLMs are trained on vast amounts of human text and encode broad commonsense knowledge about the physical world, which can be distilled into labels for the dataset.

Labeling criteria

Each candidate interaction was submitted to an LLM (*Claude*, *Anthropic*), instructed through a fixed system prompt to act as the safety supervision module of a domestic robot. The full prompt is reported in Figure 3.2 and constitutes the operational definition of the ground truth: for every interaction, the model assigns 1 if the configuration represents a severe physical hazard, and 0 otherwise.

Rather than leaving the judgment open-ended, the prompt encodes an explicit rubric. A label of 1 is restricted to four categories of physical danger: electrical and fire hazards, catastrophic mechanical damage, contamination of food-related appliances, and sharp or toxic objects accessible in spaces used by children or pets. Everything else must be labeled 0, even if the configuration is unusual, messy, or contextually strange. The prompt states this explicitly: the model is instructed that placing an object in an unexpected room is not, by itself, a danger. A knife found in a bathroom is certainly out of place, but it poses no more physical risk than the same knife in a kitchen drawer. This rubric implements the distinction drawn in Section 1.2 between statistical rarity and physical danger: the rubric forces the oracle to separate the two, so that the resulting labels, and, in turn, the model trained on them, target genuine hazards rather than mere novelty.

System prompt of the labeling oracle

You are the supervision and safety module of a smart domestic robot. Your task is to evaluate whether placing two objects together in a given room context constitutes a SAFE/NORMAL SITUATION (0) or a SEVERE ANOMALY/DANGER (1).

CRITERIA FOR 1 (SEVERE ANOMALY / UNSAFE):

Assign 1 only if the specific combination of objects causes:

- [A] CRITICAL DANGER: Electronics + Water proximity. Heat sources near flammables.
- [B] CATASTROPHIC DAMAGE: Extremely heavy objects placed ON TOP OF fragile items.
- [C] EXTREME CONTAMINATION: Non-food/dirty items INSIDE kitchen appliances.
- [D] SHARP/TOXIC HAZARD: Dangerous objects (knives, chemicals) accessible in child/pet spaces.

CRITERIA FOR 0 (SAFE / NORMAL):

Assign 0 if the action is physically safe, EVEN IF it is weird, messy, or contextually unusual.

Being in an unexpected room does NOT make something dangerous by itself.

IMPORTANT: Base your judgment on the PHYSICAL DANGER of the combination, not on whether it seems contextually appropriate. A knife in a bathroom is unusual but not inherently dangerous.

Respond EXCLUSIVELY with a JSON containing the array of labels (0 or 1). No other text.

Figure 3.2: The system prompt used to instruct the LLM oracle. The rubric restricts positive labels to four categories of physical danger ([A]–[D]) and explicitly separates physical hazard from contextual unusualness.

Labeling procedure

Each interaction is converted into a short textual description specifying the room context, the two objects with their materials, and the requested relation (e.g., “In a ‘kitchen’ setting: take ‘newspaper’ (material: paper) and do ‘placed_on_top’ with ‘stove’ (material: metal)”). To make the process efficient and reproducible, three measures were adopted. First, the labeling operates on unique interactions: when the same objects-relation-context combination appears multiple times in the dataset, its textual description is submitted to the LLM only once, and the resulting label is propagated to all matching entries. Second, interactions are processed in batches of 30 per request, with the model returning the labels in a structured JSON format. Third, every response is

stored in a persistent cache, so that re-running the pipeline does not repeat API calls and the labeling is exactly reproducible. Failed requests are retried with increasing delays.

The result of this stage is a fully labeled dataset of 8258 interactions, of which 747 (9%) are labeled as hazardous. This imbalance is expected and, in fact, desirable: it reflects the reality that most everyday placements are safe, and it matches the training strategy described in the next section, where the model learns exclusively from the safe portion of the data; hazardous examples are needed only in limited numbers, to validate the detector and calibrate its decision threshold.

3.6 Relational Graph Autoencoder

The labeled dataset could, in principle, be used to train a standard binary classifier. This approach, however, would suffer from two weaknesses. First, the hazardous class is small and cannot cover the space of possible dangers: a classifier trained on it would learn to recognize the specific hazards present in the dataset, with no guarantee of generalizing to unseen ones. Second, in a safety-critical setting it is preferable to model normality and treat any significant deviation as suspicious, rather than to enumerate what is dangerous. The framework therefore adopts the opposite strategy: a graph autoencoder is trained exclusively on safe interactions, so that it internalizes how objects normally relate to each other in a domestic environment; hazardous configurations are then detected at inference time as inputs the model fails to reconstruct correctly.

Per-room graph construction

The training data is organized into one graph per room *category*: all safe interactions taking place in a kitchen, for instance, contribute to a single “kitchen” graph, regardless of which of the three scenes they come from. Each graph is therefore a prototype of its room type rather than one specific physical room. Nodes represent the unique objects appearing in the graph, described by the 393-dimensional hybrid features of Section 3.4; each interaction contributes a directed edge from the manipulated object to the destination object. Every edge is associated with a three-dimensional one-hot vector encoding its relation type (*placed_on_top*, *placed_inside*, or *placed_near*): these vectors are not inputs to the network, but the *targets* it is trained to reconstruct.

Encoder-decoder architecture

The model follows an encoder-decoder design operating on graphs, summarized in Figure 3.3.

The **encoder** is a graph neural network composed of three GraphSAGE convolutional layers, each followed by layer normalization and a ReLU activation. Each layer updates the representation of a node by aggregating the representations of its neighbors; stacking

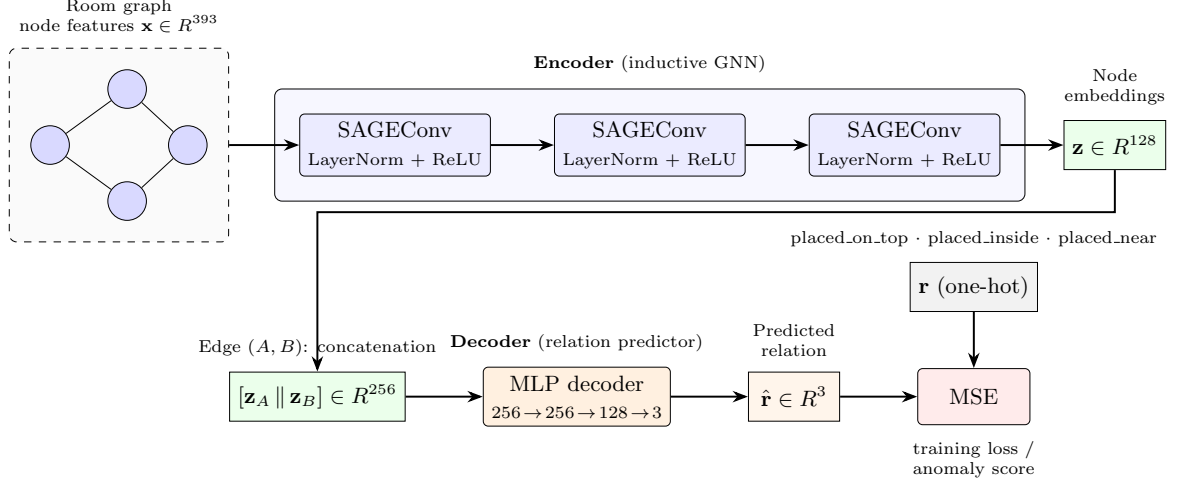


Figure 3.3: Architecture of the Relational Graph Autoencoder. The encoder maps each node of the room graph to a 128-dimensional embedding through three GraphSAGE layers; the decoder predicts the relation between a pair of nodes from the concatenation of their embeddings. The MSE between predicted and one-hot target relation serves as training loss and, at inference time, as anomaly score.

three layers means that the final embedding of a node summarizes information from its 3-hop neighborhood in the room graph. The encoder thus maps every node from its 393-dimensional input features to a 128-dimensional latent embedding that is aware of the surrounding context: the same object ends up with different embeddings depending on the room it is in and the objects it is connected to.

The choice of GraphSAGE over alternative graph convolutions is motivated by its *inductive* nature. Some graph learning methods compute a fixed embedding for each node of the training graph, and therefore cannot handle nodes that were not present during training. GraphSAGE instead learns a general rule (the aggregation function) for building a node’s embedding from its own features and those of its neighbors; since the rule does not depend on the identity of the node, it can be applied unchanged to objects never seen before. This property is essential here: at inference time, the requested object is inserted into the room graph as a new node (Section 3.7), and the model must produce a meaningful embedding for it without retraining.

The **decoder** is a relation predictor. Given a pair of nodes connected by an edge, it takes the concatenation of their two latent embeddings and processes it through a multi-layer perceptron, producing a three-dimensional output vector. The decoder is trained so that this output reproduces the one-hot encoding of the relation that actually connects the two objects: in other words, given two objects in their room context, the model learns to answer the question “how would these two objects normally be related?”.

Training strategy

Training minimizes the mean squared error (MSE) between the decoder output and the one-hot relation targets, over all edges of all room graphs. Since the training graphs are built exclusively from interactions labeled as safe, the model never observes a hazardous configuration during training: the only relational patterns it learns are those of a normal household - books lie on desks, food goes inside refrigerators, soap sits near sinks.

The intuition behind the anomaly detector follows directly. After training, the model reconstructs familiar, safe relational patterns with low error. When asked instead to reconstruct the relation of an abnormal pairing (e.g. a hair dryer inside a bathtub) the model is faced with a combination of objects and context that never appeared among the safe examples it was trained on. Its prediction therefore tends to disagree with the requested relation, and the size of this disagreement (the reconstruction error) is a natural measure of how far the requested interaction departs from domestic normality. How this quantity is computed in practice, and how it is turned into a decision, is the subject of the next section.

During training, the MSE between predicted and target relation vectors is monitored after every epoch on a held-out set of safe interactions. The learning rate is reduced when this validation error stops improving, training is halted early once no further progress is observed, and the final weights are those that achieved the lowest validation error, rather than those produced by the last epoch. The exact data splits and hyperparameter values are reported in Chapter 4.

3.7 Anomaly scoring and decision logic

The previous section described how the RGAE learns, offline, to reconstruct the relations that connect objects in safe room graphs. This section addresses the complementary, inference-time problem: given a new interaction requested by the user - one involving an object that may not even belong to the trained graph - how is an anomaly score computed, and how is that score converted into an operational decision?

Inductive scoring of unseen interactions

A key requirement of the framework is that it must be able to evaluate objects it has never encountered during training, since the semantic class of the object requested by the user is not restricted to the vocabulary of the three scenes used to build the training graphs. To this end, a candidate interaction is not evaluated by re-training or fine-tuning the model, but by *virtually augmenting* the trained room graph with the new nodes it involves, and then running a single forward pass of the already-trained encoder-decoder.

Concretely, for a requested interaction (object A , object B , relation r , room context c), the procedure retrieves the training graph associated with room c and proceeds as

follows. For each of the two objects that is not already present in that graph, its hybrid feature vector (Section 3.4) is computed on the fly, exactly as during training: a sentence embedding of the “class made of material” string, concatenated with the nine-dimensional danger profile. The new node is then anchored to the existing graph by connecting it to its $K = 3$ most similar neighbors, where similarity is measured as the cosine similarity between the new node’s feature vector and those of all nodes already in the graph. These anchor edges do not encode a specific relation, they exist only to give the GraphSAGE encoder a local neighborhood from which to propagate information to the new node, consistently with how the encoder was trained to aggregate 3-hop neighborhoods.

Once the graph has been augmented with the (at most two) new nodes and their anchor edges, the encoder is run once over the whole augmented graph to obtain updated embeddings, and the decoder is queried on the specific edge (A, B) corresponding to the requested relation r . Let $\mathbf{r}_{ij} \in \{0, 1\}^R$ denote the one-hot target vector encoding the requested relation between objects i and j (with $R = 3$, corresponding to *placed_on_top*, *placed_inside*, and *placed_near*), and let $\hat{\mathbf{r}}_{ij}$ denote the decoder’s predicted relation vector for that pair. The anomaly score is defined as the mean squared error between the two vectors, the same quantity that served as the training loss:

$$S_{\text{anomaly}} = \frac{1}{R} \sum_{k=1}^R (r_{ij,k} - \hat{r}_{ij,k})^2. \quad (3.1)$$

Intuitively, if A and B are two objects that a normal household would plausibly place in that way (e.g., a book on a desk), the decoder, having learned the manifold of safe configurations, reconstructs a relation vector close to the one-hot target, yielding a low value of S_{anomaly} . If instead the requested configuration is unlike anything observed among safe interactions (e.g., a hair dryer inside a bathtub), the decoder’s prediction diverges from the target and the score is high. Because this whole procedure only requires a forward pass, it can be executed in real time as part of the robot’s decision loop.

Threshold calibration via Youden’s J statistic

A raw value of S_{anomaly} is not directly actionable: it must be mapped to a decision through a threshold, and that threshold must be chosen so as to balance the two possible errors - letting a truly dangerous action through, versus needlessly blocking a legitimate one. The threshold is calibrated, rather than fixed by hand, on a held-out validation set that combines an equal number of safe and hazardous interactions, so that the calibration is not biased toward the majority safe class.

The calibration proceeds as follows. Every interaction in the validation set is scored with Eq. 3.1, producing a set of scores together with their ground-truth safety labels (safe or hazardous). Since it is not known in advance which cutoff best separates the two classes, every value observed in this set of scores is tried, one at a time, as a candidate decision threshold τ : for a given τ , an interaction is classified as hazardous if $S_{\text{anomaly}} >$

τ , and as safe otherwise. For each candidate, comparing the resulting safe/hazardous predictions against the ground-truth labels yields the true positive rate $\text{TPR}(\tau)$ (the fraction of hazardous interactions correctly flagged) and the false positive rate $\text{FPR}(\tau)$ (the fraction of safe interactions incorrectly flagged). Repeating this for every candidate threshold produces a sequence of $(\text{FPR}(\tau), \text{TPR}(\tau))$ pairs, one per threshold; plotting these pairs is, by definition, the ROC curve of the detector.

Among all thresholds on this curve, the one adopted as the operating point is the one that maximizes Youden’s J statistic,

$$J(\tau) = \text{TPR}(\tau) - \text{FPR}(\tau), \quad (3.2)$$

which corresponds to the point of the ROC curve farthest from the diagonal of random guessing. The resulting value, denoted τ^* and referred to as the *global threshold*, is then reused, unmodified, across all room contexts.

Three-tier decision policy

A single hard threshold would force every interaction into a binary safe/unsafe outcome, which is undesirable in an HRI setting: some configurations are only mildly atypical, and treating them identically to overtly dangerous ones would make the robot needlessly rigid and erode user trust. The framework therefore derives a second, lower boundary from the global threshold, defined as a fixed margin below it (twenty percent of its value), $\tau_{\text{warn}} = \tau^* - 0.2\tau^*$, and uses the two boundaries to partition the anomaly score into three regions:

- **Autonomous approval (green zone):** if $S_{\text{anomaly}} \leq \tau_{\text{warn}}$, the requested configuration closely matches patterns of domestic normality; the robot executes the action without further confirmation.
- **Human-in-the-loop warning (grey zone):** if $\tau_{\text{warn}} < S_{\text{anomaly}} \leq \tau^*$, the interaction is atypical but not clearly hazardous; the robot does not execute the action autonomously and instead requests confirmation from the user before proceeding.
- **Automatic blocking (red zone):** if $S_{\text{anomaly}} > \tau^*$, the configuration is classified as a genuine hazard; the robot refuses to execute the action and reports the refusal to the user.

This three-tier scheme is deliberately conservative in its middle region: rather than committing to a binary judgment exactly at τ^* , the robot defers to the human whenever the evidence is ambiguous, reserving fully autonomous refusal for scores that clearly exceed the calibrated threshold. As will be discussed in Chapter 5, this same three-tier structure is what allows the decision policy to be further modulated by the profile of the user issuing the command, tightening the grey-zone response from a mere warning to an outright denial when the requester is identified as a child.

Chapter 4

Experimental analysis and offline results

4.1 Training setup

The dataset resulting from the pipeline of Chapter 3 comprises 8,258 labeled interactions, of which **91.0%** are **safe** and **9.0%** are **hazardous**, consistent with the expected rarity of genuine physical danger in domestic environments (Figure 4.1 left). Interactions are dominated by *placed_near* (3,921 samples), followed by *placed_on_top* (2,394) and *placed_inside* (1,943), and are unevenly distributed across room types, with kitchen and bathroom contexts contributing the largest share of samples; this is not surprising, since these rooms combine the highest object density with the largest number of electrical- and water-related items.

Breaking this composition down further, Figure 4.2 (left) shows that hazardous interactions are concentrated in the kitchen and bathroom contexts, coherently with the electrical/water hazard categories privileged by the labeling rubric, while the material-level breakdown (right) reveals two complementary effects. In absolute terms, the safe class is dominated by the materials that are simply most frequent in the dataset (unknown, ceramic, wood, and paper). In relative terms, however, the fraction of hazardous interactions varies markedly across materials: while 9% of all interactions in the dataset are labeled hazardous, this fraction rises to roughly 20% for interactions involving metal objects and 37% for glass ones. This is coherent with the labeling rubric of Section 3.5: metallic objects are typically electrical or heat-related appliances (category [A]), while glass objects are fragile items exposed to catastrophic mechanical damage (category [B]). The figure for glass should nevertheless be read with caution, as it is estimated from only about thirty samples.

The RGAE was trained exclusively on the safe partition (60/20/20 train/val/test split), using three GraphSAGE layers (*hidden size 128*), an MLP decoder ($256 \rightarrow 256 \rightarrow 128 \rightarrow 3$, *dropout 0.2*), Adam optimizer ($lr = 1 \times 10^{-3}$, *weight decay* 10^{-4}), and a ReduceLROn-

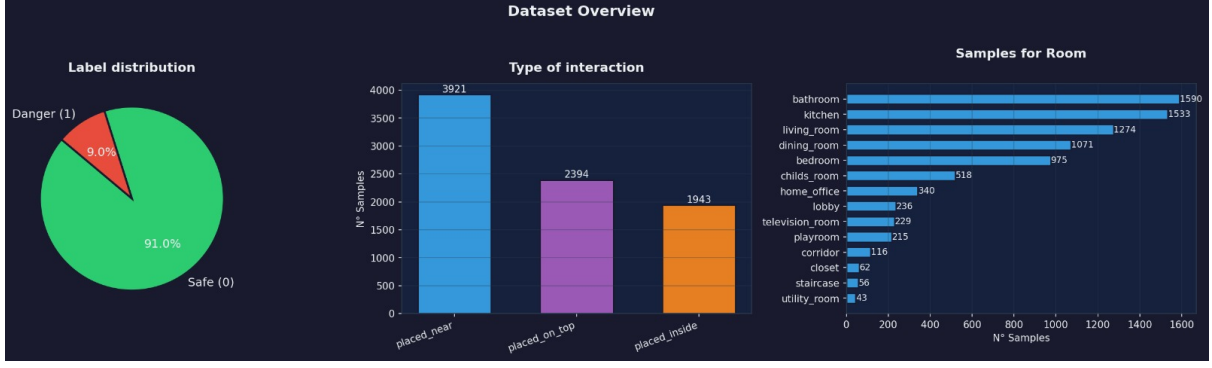


Figure 4.1: Dataset composition: label distribution, interaction types, and per-room sample counts.

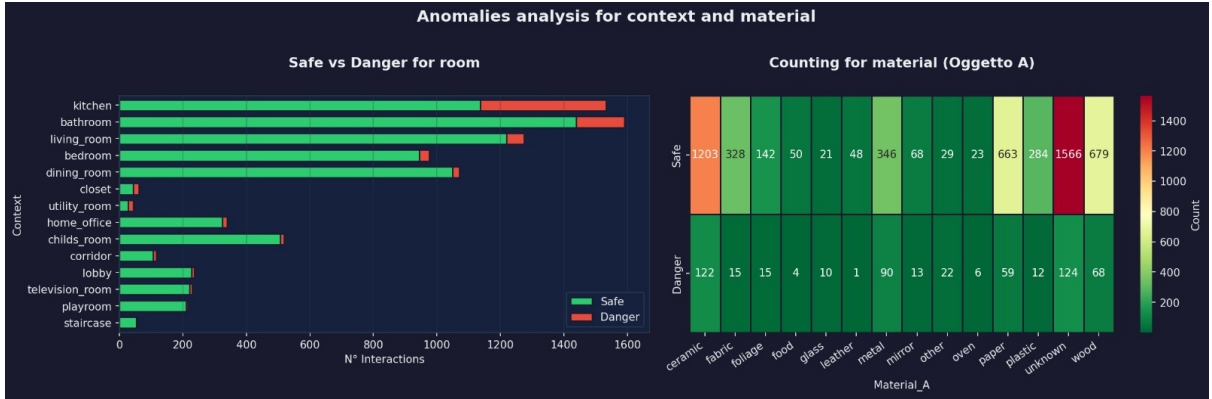


Figure 4.2: Distribution of safe vs. hazardous interactions by room context and by material

Plateau scheduler (*factor 0.5*, *patience 20*). Training ran for a maximum of *300 epochs* with early stopping (*patience 30*) on the validation MSE. As shown in Figure 4.3, the validation loss reaches its minimum at epoch 14, after which it plateaus and slowly increases while the training loss keeps decreasing, which is a typical overfitting signature that justifies selecting the epoch-14 checkpoint rather than the final one. The step-wise learning-rate reduction visible around epoch 35 (from 10^{-3} to 5×10^{-4}) occurs exactly 20 epochs (the scheduler patience) after the best validation epoch; since the reduced learning rate did not restore any improvement, early stopping halted training at epoch 44, 30 epochs after the optimum. The corresponding decision threshold (0.278) was calibrated on a balanced validation set via **Youden’s J index**, maximizing TPR - FPR on the ROC curve.

4.2 Anomaly score evaluation

Applying the calibrated threshold to the held-out balanced test set yields the metrics summarized in Figure 4.4: an **AUC of 0.854** and a **PR-AUC of 0.874**, with a confusion matrix of *321 true safe*, *55 false alarms*, *107 missed hazards*, and *269 correctly detected hazards* corresponding to a **precision of 0.83**, a **recall of 0.72**, and an **F1-**

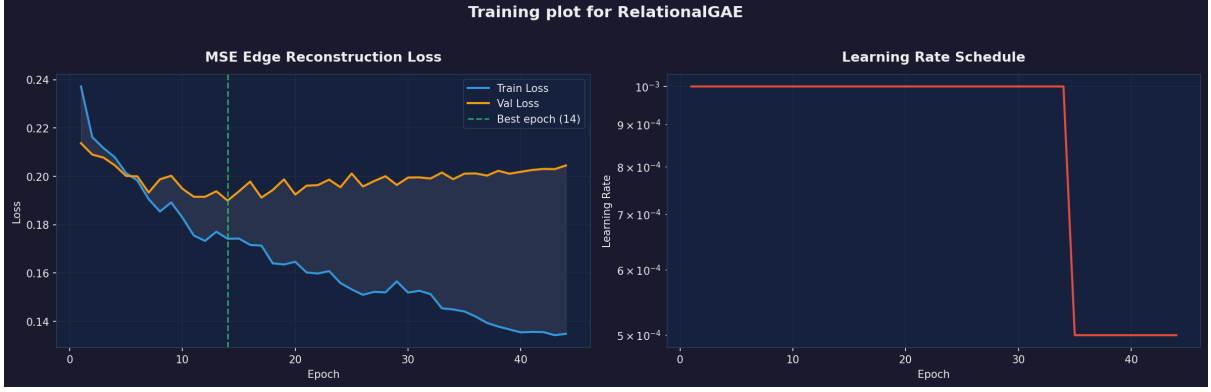


Figure 4.3: MSE reconstruction loss (train/validation) and learning-rate schedule over training.

score of 0.77. The anomaly-score histogram confirms that safe interactions concentrate near zero error while hazardous ones shift toward higher values, though the two distributions overlap noticeably around the threshold, which is the direct source of both the false alarms and the missed detections discussed in Section 4.4.

A complementary, more interpretable validation was performed with a handcrafted bench of 16 interactions covering unambiguous safe/dangerous cases and deliberately ambiguous “edge cases” (Figure 4.5), where each score is compared against both the global threshold τ^* and the warning threshold τ_{warn} , so that every request falls into one of the three decision bands of Section 3.7. The model assigns its highest score (0.449) to shoes placed inside an oven and correctly blocks other clear hazards (bleach inside a refrigerator, 0.324; a hair dryer inside a bathtub, 0.287), while also generalizing to unseen ambiguous configurations such as a towel inside an oven (0.340) or dumbbells on a glass table (0.293). Some benign but mildly atypical requests, such as a book placed on a desk (0.250), fall instead in the intermediate warning band: rather than being misclassified outright, they trigger a confirmation request, which is precisely the behavior the grey zone was designed for. Two notable misses stand out, both falling in the safe band: newspaper placed on a stove (0.183), designed as a combustion hazard, and toaster placed near the sink (0.037), which even receives the lowest score of the entire bench despite coupling an electrical appliance with a water source. Both are concrete instances of the recall gap already visible in Figure 4.4, and their causes are analyzed in Section 4.4.

4.3 Explainability and spatial visualization

Figure 4.6 gives a direct visual account of the graphs on which the RGAE was trained. Each panel shows one room-type graph (bathroom, bedroom, child’s room), with nodes representing the unique objects and edges colored by relation type. Note that a node’s room label reflects the destination context of a safe interaction rather than the object’s everyday location. The visualization confirms two properties anticipated in Section 4.1:

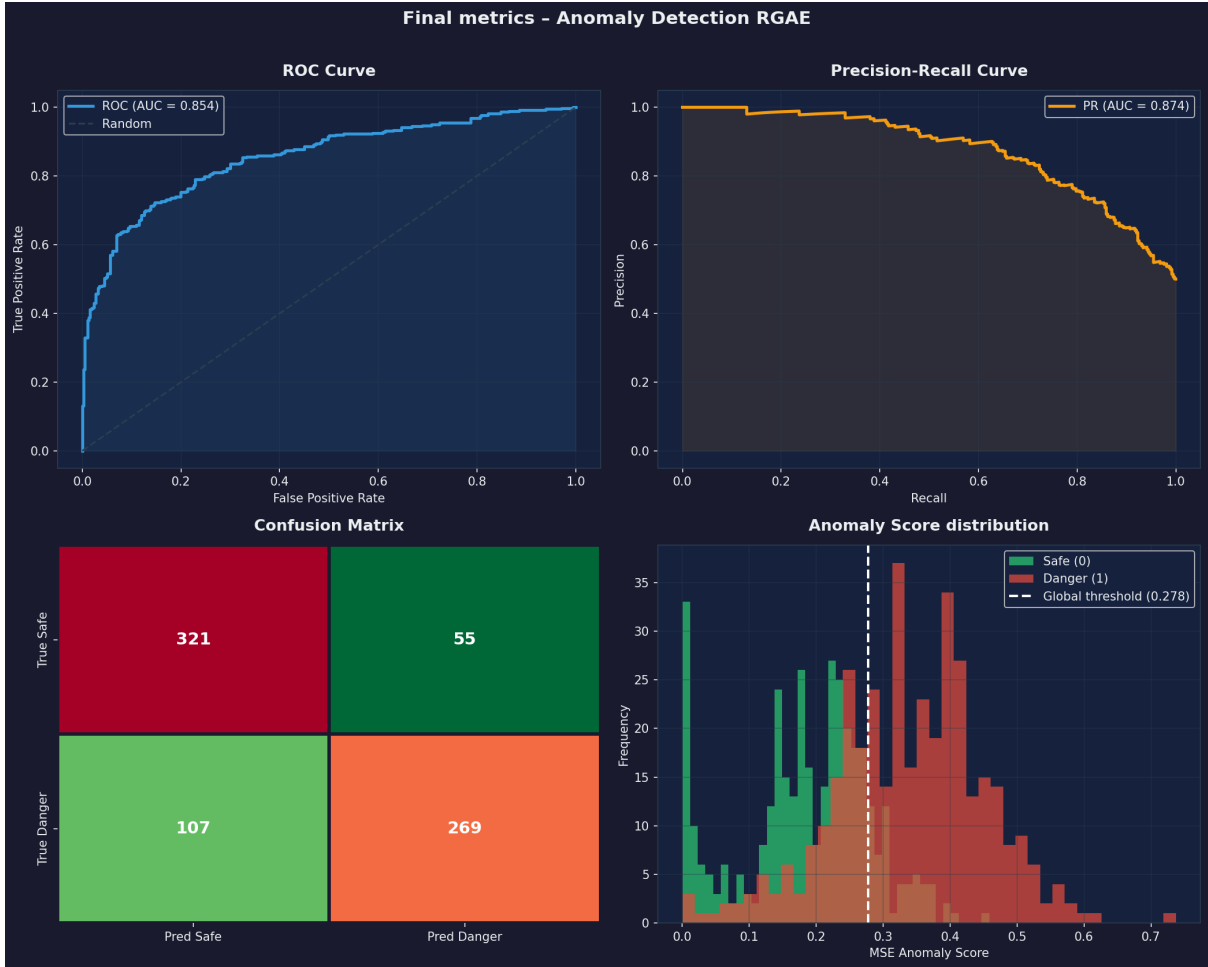


Figure 4.4: ROC curve, precision–recall curve, confusion matrix, and anomaly-score distribution on the test set.

graph density scales with object variety, in fact the bathroom graph is both the largest (83 nodes) and the densest (291 edges), and *placed_near* dominates numerically over the other two relations in every room, consistent with the overall interaction-type distribution of Figure 4.1. The child’s room graph, notably smaller and sparser, reflects the more limited object vocabulary typical of that context, which is also relevant to the child-specific access-control policy discussed in Chapter 5.

To assess whether the model’s decisions can be justified rather than treated as a black box, Figure 4.7 reports two complementary analyses. First, as shown by the Pearson correlation between each attribute of the combined danger profile and the anomaly score, the physical features that most readily trigger the model’s alarm are those related to electrical (0.29) and thermal (0.26) components. This behavior demonstrates that the architecture has successfully assimilated the strict safety criteria distilled by the LLM during the data labeling phase; since hazards linked to electricity and heat represent the most catastrophic threats in a domestic environment, the model correctly established them as the primary drivers of the anomaly score. Second, translating the score distribution into the three-tier policy of Chapter 5 (safe / warning / danger zones) yields an

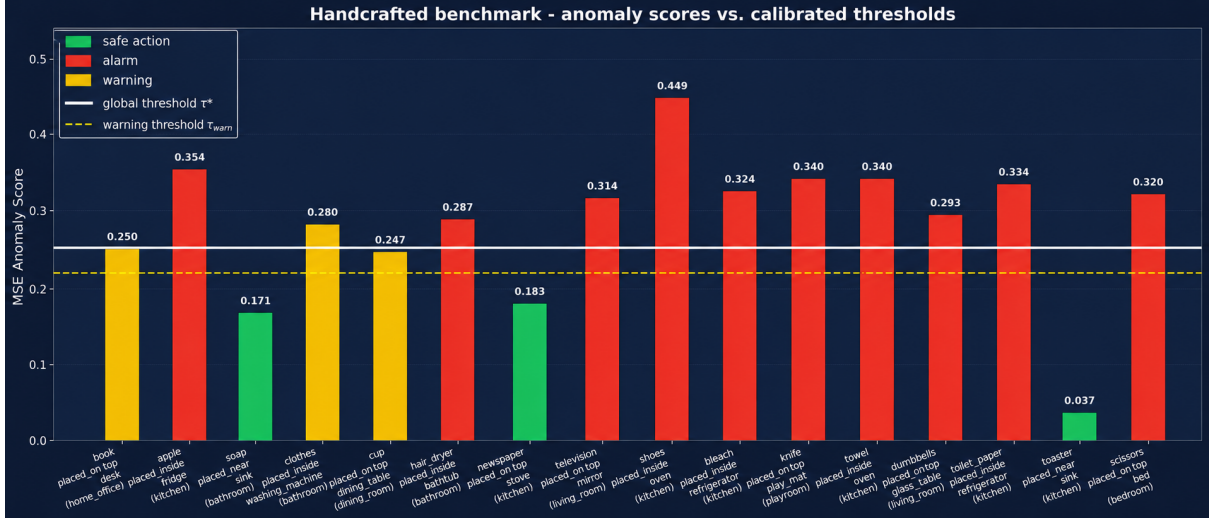


Figure 4.5: Anomaly scores for a handcrafted bench of safe, dangerous, and edge-case interactions, compared against the calibrated global threshold τ^* (white) and the warning threshold τ_{warn} (yellow dashed); bar colors reflect the three decision bands of Section 3.7.

autonomous-approval rate of **30.9%**, a human-in-the-loop rate of **20.1%**, and a blocking rate of **49.1%** on the balanced test set.

4.4 Limitations of the offline model

The results above confirm the validity of the proposed approach, while also pointing to some interesting directions for refinement. The recall of 0.72 shows that the majority of hazardous interactions are correctly identified, though a portion is still missed, and the two misses highlighted in Section 4.2 expose the two mechanisms behind this gap. The first is attribute-related: as shown in Figure 4.7, the flammable attribute exhibits a weakly negative correlation (0.14) with the anomaly score, meaning that, unlike electrical and heat-related attributes, flammability alone does not raise the model’s alarm; this explains why a combustion hazard such as the newspaper on the stove goes undetected. The second is relation-related: the toaster near the sink couples an electrical device with a water source, yet it is reconstructed almost perfectly. The likely cause is that *placed_near* is by far the most frequent relation in the training data (3,921 of 8,258 interactions) and the least constrained of the three primitives, so the decoder learns to reconstruct it with low error almost regardless of the objects involved; loose proximity hazards are therefore systematically harder to flag than containment ones. Combustion- and proximity-related dangers could thus benefit from additional emphasis in future iterations, for instance by reweighting the flammable attribute or by balancing the relation types during training. The decision threshold currently used is a single global value shared across all room contexts; since “normality” is naturally somewhat room-specific, a per-context calibration

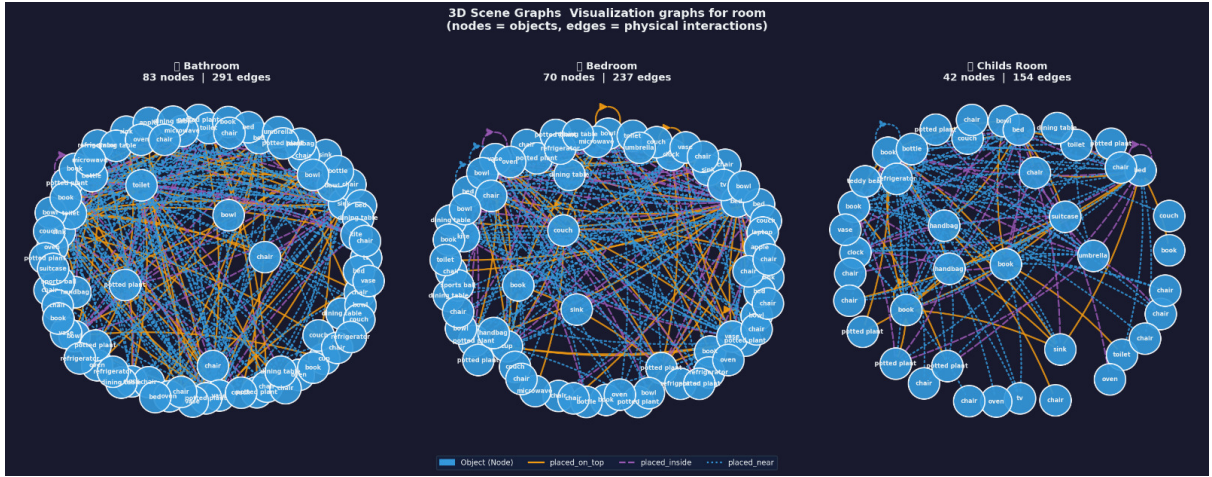


Figure 4.6: Room-level scene graphs used to train the RGAE (bathroom, bedroom, child’s room), with nodes as objects and edges colored by relation type

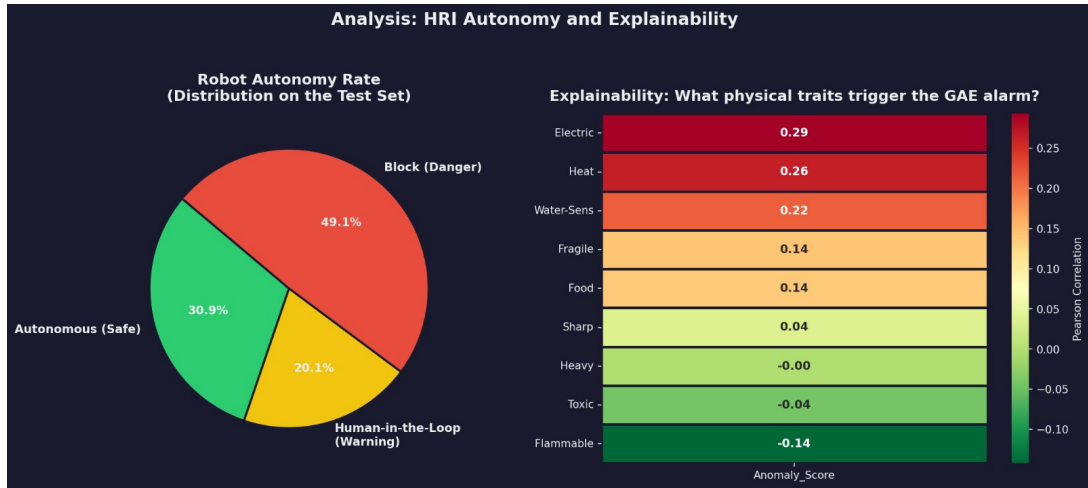


Figure 4.7: Robot autonomy-rate distribution and correlation between danger-profile attributes and anomaly score.

represents a promising next step, which was not pursued here due to the limited number of labeled hazards available per individual room. The danger profile is currently encoded as binary attributes, without an explicit notion of severity: introducing graded intensity levels (for instance, distinguishing a mildly heavy object from an extremely heavy one) could further sharpen the model’s sensitivity to attributes like heavy or sharp. Finally, the current dataset is built from three static scenes and three interaction primitives, and its ground-truth labels rely on a single LLM judgment per interaction; extending the scene coverage and cross-validating a subset of labels against human annotators are natural next steps to further strengthen the pipeline in future iterations.

Chapter 5

Integration into a Human-Robot Interaction loop

5.1 From an offline detector to an embodied system

The previous chapters developed and evaluated the anomaly detector as a self-contained, offline component: given a candidate interaction, it returns an anomaly score and a safe/warning/danger verdict. For this capability to be useful, however, the detector must live inside the decision loop of a robot that talks to a human, receives a request in natural language, decides *before* acting, and communicates its decision in an understandable way.

This chapter describes that integration. The system is implemented as two ROS 2 nodes running on a TIAGo service robot in a laboratory environment. Throughout, the guiding principle is the one introduced in Chapter 1: the burden of hazard assessment is shifted from the human onto the robot, and the robot’s compliance is adapted to who is issuing the command.

5.2 System architecture and information flow

The two nodes communicate through two string topics, as shown in Figure 5.1. The `voice_interface` node is the interface with the human: it captures the request, transforms it into a structured query, and speaks the robot’s answer. The `anomaly_detector` node wraps the trained model: it subscribes to the query topic, runs the inductive inference of Section 3.7, and publishes the response.

A single exchange proceeds as follows. The voice node assembles a six-field message (the manipulated object and its material, the destination object and its material, the interaction primitive, and the room context) and publishes it on `/tiago/scene_graph_perception`. The detector node processes the message and evaluates it exactly as described in Section 3.7: objects not already present in the room graph are embedded from their class, material and danger profile, and anchored to their most similar nodes; objects the room

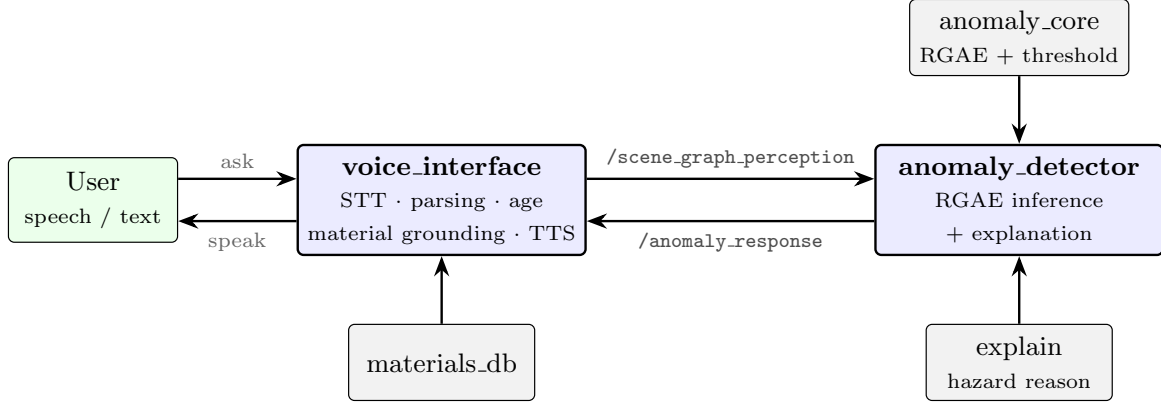


Figure 5.1: ROS 2 architecture of the conversational system. The `voice_interface` node turns a spoken request into a structured query published on `/tiago/scene_graph_perception`; the `anomaly_detector` node replies on `/tiago/anomaly_response` with a single natural-language sentence fusing verdict and justification. Supporting modules provide material grounding and hazard explanation.

already contains are reused rather than duplicated; the requested interaction is added as an edge, and the anomaly score is the reconstruction error between the predicted and the requested relation for that specific edge. Each query is evaluated on a temporary copy of the trained room graph: the requested objects and their anchor edges are added to the copy, which is discarded once the score is computed, while the stored graph is never modified. As a consequence, the outcome of a request does not depend on the requests that preceded it: the same query always yields the same verdict, which makes demonstrations repeatable.

This design also explains a structural guardrail: the inference is necessarily carried out against the graph of a room category seen during training, so a request concerning an unknown room *cannot* be evaluated. Rather than guessing, the detector answers with a dedicated sentence stating that it does not know the room.

The verdict travels back on `/tiago/anomaly_response` as a single natural-language sentence in which decision and justification are fused, and which is synthesized into speech by the voice node.

5.3 From speech to a structured query

The voice node converts free speech into the structured query the detector expects, without relying on any language model at run time, consistent with the design principle of distilling commonsense offline (Chapter 2) rather than consuming it on every request. Speech is transcribed locally with a compact speech-to-text model, and the resulting sentence is parsed by deterministic rules. The parser locates the room, an action verb, and a spatial keyword (*in*, *on top of*, *near*, ...) that acts as the boundary between the object

being moved and the destination object; each side is then matched against the system’s object vocabulary, longest phrase first, so that “*dining table*” is recognized before “*table*”. Spoken aliases are normalized to canonical names (*television* → *tv*, *fridge* → *refrigerator*), so that the many ways a user may refer to the same object all map to a single entry of the object vocabulary and of the class-to-material table. An object that matches nothing in the vocabulary is passed through as-is, so the system remains usable outside its curated vocabulary. If no room is mentioned, the robot assumes the living room and says so aloud. A sentence such as “*put the hair dryer in the bathtub in the bathroom*” is thus mapped to the query (*hair drier*, *bathtub*, *placed_inside*, *bathroom*).

A specific difficulty is the grounding of *materials*. The detector needs the material of both objects (Section 3.4), yet requiring the user to state it (e.g. “put the *plastic* hair dryer in the *ceramic* bathtub”) would be unnatural. The material is instead treated as objective knowledge the robot is expected to possess, resolved through a three-step fallback. If the user happens to mention a material, it is used; otherwise the object name is looked up in a shared class-to-material table covering the demonstration vocabulary; only for an object absent from the table the robot asks the user, as a last resort. In the common case, therefore, the material is grounded silently and the interaction remains natural, which closes the limitation left open in Chapters 3 and 4.

5.4 Context-aware access control

A central tenet of this work is that the appropriate response to a risky request depends not only on the action but also on the user issuing it. Before any request is processed, the robot asks the user’s age and derives a binary profile, *adult* or *child* (threshold: 18 years). If the age cannot be understood after repeated attempts, the system defaults to the more restrictive child profile.

This profile modulates the three-tier decision policy of Section 3.7, as summarized in Table 5.1. Safe actions are always executed and clearly dangerous ones are always refused, independently of the profile; the two profiles differ only in the intermediate, ambiguous band. For an adult, a warning triggers a human-in-the-loop exchange: the robot voices its concern, asks for explicit confirmation, and, if the user confirms, proceeds while asking to be supervised during execution (“*I will proceed, but please supervise me*”). For a child, the same warning is instead converted into a firm refusal, phrased so as not to suggest an alternative (“*No, I cannot do that. It is not safe. Please ask an adult for help*”). In this way the grey zone, which for an adult is an opportunity for shared decision-making, becomes a protective barrier for a child.

A limitation of the current policy must be acknowledged openly: the profile is self-declared and requested once per session, so a child could misstate their age or restart the interface. Robust age verification (e.g., from voice or facial cues) is a separate perception problem, deliberately out of the scope of this project and listed among its future

Anomaly band	Adult	Child
Safe (green)	execute	execute
Warning (grey)	confirm + supervise	refuse
Danger (red)	refuse	refuse

Table 5.1: Context-aware access control. The user profile changes the robot’s behavior only in the intermediate warning band, tightening a confirmation into an outright refusal for a child.

developments.

5.5 Verbal hazard explanation

Refusing or questioning a command is of little use in HRI if the robot cannot say *why*. Because the RGAE returns a single scalar score rather than a symbolic cause, the system pairs it with a dedicated explanation module that reconstructs the most plausible reason for a flagged interaction. Given the two objects, their properties, the interaction primitive, and the room, the module checks a small ordered set of hazard patterns (electrical device near water, flammable item on a heat source, heavy object on a fragile support, sharp object in a child area, toxic substance near food, and so on) and returns a natural spoken sentence for the first pattern that applies, for example *“the hair dryer is an electrical device, and the bathtub may contain water; together they create a serious risk of electric shock”*. When the detector flags an interaction that none of the patterns explain, the module falls back to an honest, neutral message stating that the configuration is atypical and cannot be assumed safe.

For the purpose of *explanation* only, this module exploits a lexical knowledge base richer than the minimal keyword set used to build the model’s danger profiles: it recognizes, for instance, that a *hair dryer* is an electrical appliance even though that specific term is absent from the profile keywords. This affects only the wording of the justification, never the detector’s decision, which remains entirely determined by the learned model. The separation is deliberate: the neural component decides, while transparent rule sets verbalize the decision, combining the generalization of the former with the intelligibility required for user trust.

5.6 Example interactions

Having described the individual components of the loop, this section shows the system in operation through a set of recorded interactions. Its aim is qualitative rather than

quantitative: to make concrete how a spoken request is turned into a verdict and, above all, how the same request can be treated differently depending on who issues it.

At the beginning of a session the robot greets the user and asks for their age, from which it derives the binary *adult/child* profile. This question is asked only once and the profile is retained for the remainder of the session. From then on the user issues natural-language commands; each command is transcribed locally, parsed into the four-field query (*object A*, *object B*, *primitive*, *room*) of Section 5.3, and evaluated by the detector, which replies with a single sentence fusing the verdict with its justification. When a required material is unknown, the robot asks for it before deciding, and when an action falls in the warning band the response depends on the profile, as illustrated below.

The exchanges reported in this section are verbatim transcriptions of recorded runs of the system on the real TIAGo robot in the laboratory. Throughout, **Robot:** marks a sentence spoken by the robot, while **Children:** and **Adult:** mark sentences spoken by the human user. They are organized as two sessions: a first session with a child (Experiments 1–3) and a second session with an adult (Experiments 4–8).

Experiment 1: child, scissors near the bed (warning → refusal).

Robot: Hello, before we start, how old are you?

Children: I'm 10 years old.

Robot: Thank you. Since you are 10, I will refuse any risky action.

Children: Put the scissors near the bed in the bathroom.

Robot: No, I cannot do that. It is not safe. Please ask an adult for help.

Experiment 2: child, book inside the oven (danger).

Children: Put the book inside the oven in the kitchen.

Robot: No, I will not do that. The book can catch fire and the oven gets hot; this could start a fire in the kitchen.

Experiment 3: child, toy near the bed (safe).

Children: Put the toy near the bed in the bedroom.

Robot: What is the toy made of?

Children: It is made of wood.

Robot: Yes, I can do that. The action looks safe.

Experiment 4: adult, scissors near the bed (warning → confirmation).

Robot: Before we start, how old are you?

Adult: I'm 25 years old.

Robot: Thank you. You are 25, so I will ask for your confirmation on risky

actions.

Adult: Put the scissors near the bed in the bedroom.

Robot: I have a concern: the scissors are sharp, and leaving them near the bed means someone could get hurt. Do you confirm that I should proceed?

Adult: Yes.

Robot: Understood. I will proceed, but please supervise me.

Experiment 5: adult, laptop inside the bathroom sink (warning → confirmation).

Adult: Put the laptop inside the sink in the bathroom.

Robot: I have a concern: the laptop is an electrical device and the sink may contain water; together they create a serious risk of electric shock. Do you confirm that I should proceed?

Adult: Yes.

Robot: Understood. I will proceed, but please supervise me.

Experiment 6: adult, bottle near the microwave (safe).

Adult: Put the bottle near the microwave in the kitchen.

Robot: Yes, I can do that. The action looks safe.

Experiment 7: adult, bottle inside the microwave (danger).

Adult: Put the bottle inside the microwave in the kitchen.

Robot: No, I will not do that. A bottle is not normally placed inside a microwave in the kitchen, so I cannot be sure it is safe.

Experiment 8: adult, television on a cup (danger).

Adult: Put the TV on the cup in the living room.

Robot: What is the cup made of?

Adult: It is made of ceramic.

Robot: No, I will not do that. The TV is too heavy for the cup, which is fragile and could shatter.

Same action, different treatment: Experiment 1 versus Experiment 4. Experiments 1 and 4 request the identical action and receive the identical internal assessment: the detector places the scissors-near-bed interaction in the warning band. What differs is only the user profile, and the responses follow exactly the policy of Table 5.1: the adult is granted autonomy through confirmation, the child is protected through denial. The grey zone thus behaves, in practice, as designed: an opportunity for shared decision-making for one user, a protective barrier for the other.

Same objects, different relation: Experiment 6 versus Experiment 7. Experiments 6 and 7 form a second instructive pair, complementary to the previous one: here the user and the objects are identical, and what changes is only the requested spatial relation. Placing the bottle near the microwave is approved as safe, while placing the same bottle inside it is refused. This contrast makes visible a property of the detector that no single experiment could show: the anomaly score is not a function of the object pair alone, but of the full (object, object, relation, context) tuple. The model has not learned that bottles and microwaves are incompatible; it has learned that their proximity is part of ordinary kitchen life, whereas their containment is not, a sealed bottle heated in a microwave can in fact burst. In other words, the relational structure of the scene graph is doing real work in the decision: the same pair of nodes yields opposite verdicts depending on the edge that connects them, which is precisely the kind of relation-level reasoning that motivated the graph-based formulation of Chapter 3 over object-level approaches such as those discussed in Chapter 2.

Chapter 6

Conclusions and future work

This report presented a framework for semantic hazard detection in HRI, aimed at enabling a domestic robot to assess the safety of a user-requested action before executing it. Rather than limiting safety to collision avoidance or motion planning, the proposed approach reasons about the physical consequences of object placements, allowing the robot to recognize potentially dangerous situations and prevent unsafe actions before they occur.

The proposed solution combines relational scene representations, graph neural networks, and commonsense knowledge distilled from an LLM into a unified anomaly detection framework. Starting from indoor scene graphs extracted from the Stanford 3D Scene Graph dataset, candidate object interactions were generated and automatically labeled according to their physical safety. These data were then used to train a Relational Graph Autoencoder capable of learning the relational structure of safe domestic environments and identifying hazardous configurations through reconstruction error. The resulting detector was subsequently integrated into an HRI pipeline in which user commands are evaluated, through virtual insertion into the room’s scene graph, before execution, and translated into one of three possible responses: autonomous approval, user confirmation, or action refusal.

The experimental evaluation shows that learning a model of normal domestic interactions provides an effective strategy for semantic hazard detection. Despite relying on only three indoor environments for training, the proposed framework successfully identifies the majority of dangerous configurations while maintaining the ability to generalize to previously unseen object combinations. Moreover, deploying the detector within a conversational HRI architecture on a real TIAGo robot demonstrates how anomaly detection can become an active component of robotic decision making, allowing the robot not only to reject unsafe requests but also to communicate the reasons behind its decisions in a transparent and understandable way, adapting its degree of compliance to the profile of the user issuing the command.

Several directions naturally emerge for future work. On the detection side, extending the framework to a larger number of environments, object categories, and spatial

relations would improve both its coverage and its ability to model increasingly complex manipulation scenarios. The current binary danger profile could be replaced by richer representations capable of expressing different levels of hazard severity, while context-dependent anomaly thresholds could further improve the balance between safety and robot autonomy; reweighting attributes such as flammability and rebalancing the relation types during training would additionally address the specific failure modes identified in Section 4.4. Complementing the LLM-generated labels with human annotations would also provide a useful validation of the supervision process.

On the interaction side, two developments follow directly from the current system. First, the self-declared user profile could be replaced by robust age estimation from voice or facial cues, closing the loophole acknowledged in Section 5.4. Second, and most importantly, the present system implements the conversational safety loop: the verdict is communicated verbally, but not yet followed by physical execution. The natural next step is therefore full embodiment: after an approved action, the robot would navigate to the relevant room and verify perceptually, through its head camera and an object detector, that the objects involved in the request are actually present and reachable. Such a perceptual check would complement the detector’s reasoning, pairing its judgment on whether an action *would be* safe with a confirmation of whether it is *actually realizable*, and would move the system from judging requested configurations to acting upon them in a real household.

Bibliography

- [1] M. Ahn et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.
- [2] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese. 3d scene graph: A structure for unified semantics, 3d space, and camera. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5663–5672, 2019.
- [3] H. Kim, B. S. Lee, W.-Y. Shin, and S. Lim. Graph anomaly detection with graph neural networks: Current status and challenges. *IEEE Access*, 10:111820–111829, 2022.
- [4] A. Leanza, A. Moroncelli, G. Vizzari, F. Braghin, L. Roveda, and B. Spahiu. Concept-bot: Enhancing robot’s autonomy through task decomposition with large language models and knowledge graph. arXiv preprint arXiv:2509.00570, 2025.
- [5] Z. Song, G. Ouyang, M. Fang, H. Na, Z. Shi, Z. Chen, Y. Fu, Z. Zhang, S. Jiang, M. Fang, L. Chen, and X. Chen. Hazards in daily life? enabling robots to proactively detect and resolve anomalies. arXiv preprint arXiv:2411.00781, 2024.